1. What is an array in C?
- **(A)** Homogeneous contiguous data
- **(B)** Heterogeneous linked list
- **(C)** Dynamic tree node
- **(D)** Primitive scalar type

2. What is the index of the first element in any standard C array?
- **(A)** 0 (zero-indexed)
- **(B)** 1
- **(C)** -1
- **(D)** Depends on declaration

3. For an array declared as int data[10]; what is the valid index range?
- **(A)** 0 to 9
- **(B)** 1 to 10
- **(C)** 0 to 10
- **(D)** -1 to 9

4. How are array elements laid out in physical computer RAM?
- **(A)** Contiguous linear blocks
- **(B)** Scattered in heap
- **(C)** Random hash buckets
- **(D)** Non-linear nodes

5. What is the memory address formula for element arr[i] of type T?
- **(A)** Base + i * sizeof(T)
- **(B)** Base + i
- **(C)** Base + sizeof(T) / i
- **(D)** Base * i

6. If base address of int arr[10] is 2000 and int is 4 bytes, what is address of arr[3]?
- **(A)** 2012 (2000 + 3*4)
- **(B)** 2003
- **(C)** 2016
- **(D)** 2008

7. What value do unspecified trailing elements receive in partial array initialization: int a[5] = {1, 2};?
- **(A)** 0 (zero)
- **(B)** Garbage values
- **(C)** 1
- **(D)** -1

8. What does the declaration int arr[5] = {0}; accomplish?
- **(A)** Initializes all 5 to 0
- **(B)** Initializes only arr[0]
- **(C)** Syntax compilation error
- **(D)** Allocates empty array

9. What happens if an uninitialized local array is read without assigning values?
- **(A)** Contains garbage values
- **(B)** Contains all zeros
- **(C)** Compiler runtime fault
- **(D)** Automatically NULL

10. Does the standard C compiler perform runtime array bounds checking?
- **(A)** No bounds checking
- **(B)** Yes (throws exception)
- **(C)** Only in debug mode
- **(D)** Only for floats

11. What type of behavior occurs when accessing an array beyond its declared size?
- **(A)** Undefined behavior
- **(B)** Compilation error
- **(C)** Array auto-resizes
- **(D)** Index wraps modulo N

12. In C, how are multi-dimensional arrays laid out in memory?
- **(A)** Row-major order
- **(B)** Column-major order
- **(C)** Diagonal-major order
- **(D)** Z-order curve

13. In a 2D array matrix[R][C], what does R represent?
- **(A)** Total rows
- **(B)** Total columns
- **(C)** Bytes per row
- **(D)** Matrix determinant

14. In row-major 2D array arr[ROWS][COLS], what is the address formula for arr[i][j]?
- **(A)** Base + (i*COLS + j)*size
- **(B)** Base + (j*ROWS + i)*size
- **(C)** Base + (i + j)*size
- **(D)** Base + (i*j)*size

15. How many elements are allocated by the declaration: double matrix[4][5];?
(A) 20 elements
(B) 9 elements
(C) 40 elements
(D) 16 elements

16. What is the memory footprint in bytes of int table[3][4] on a 32-bit/64-bit int system?
(A) 48 bytes (3*4*4)
(B) 12 bytes
(C) 24 bytes
(D) 96 bytes

17. What is the term for searching an element sequentially from the first to the last item?
(A) Linear Search
(B) Binary Search
(C) Hash Search
(D) Interpolation Search

18. What is the worst-case time complexity of Linear Search on an array of size N?
(A) O(N)
(B) O(log N)
(C) O(1)
(D) O(N^2)

19. What is the best-case time complexity of Linear Search?
(A) O(1) [First element]
(B) O(N)
(C) O(log N)
(D) O(N/2)

20. What prerequisite condition MUST be satisfied before performing Binary Search?
(A) Array must be sorted
(B) Array must have no zeros
(C) Array size power of 2
(D) Array must be float

21. What is the worst-case time complexity of Binary Search on a sorted array of size N?
(A) O(log N)
(B) O(N)
(C) O(N log N)
(D) O(1)

22. Which simple sorting algorithm repeatedly compares adjacent elements and swaps them if out of order?
(A) Bubble Sort
(B) Selection Sort
(C) Insertion Sort
(D) Merge Sort

23. What is the worst-case time complexity of standard Bubble Sort?
(A) O(N^2)
(B) O(N)
(C) O(N log N)
(D) O(log N)

24. Which sorting algorithm repeatedly finds the minimum element and moves it to the sorted boundary?
(A) Selection Sort
(B) Bubble Sort
(C) Insertion Sort
(D) Quick Sort

25. What is the time complexity of Selection Sort in both best and worst cases?
(A) O(N^2)
(B) O(N)
(C) O(N log N)
(D) O(1)

26. Which sorting algorithm builds the sorted array one item at a time, efficient for nearly sorted data?
(A) Insertion Sort
(B) Selection Sort
(C) Bubble Sort
(D) Heap Sort

27. What is the best-case time complexity of Insertion Sort on an already sorted array?
(A) O(N)
(B) O(N^2)
(C) O(log N)
(D) O(1)

28. When passing a 1D array to a function, what does the array identifier decay into?
(A) Pointer to first element
(B) Entire array copy
(C) Array structure copy
(D) Memory descriptor

29. When declaring a 2D array parameter in a function, which dimension MAY be omitted?
- **(A)** First dimension (rows)
- **(B)** Second dimension (cols)
- **(C)** Both dimensions
- **(D)** Neither dimension

30. Which function prototype correctly accepts a 2D array with 10 columns?
- **(A)** void f(int arr[][10])
- **(B)** void f(int arr[10][])
- **(C)** void f(int arr[][])
- **(D)** void f(int *arr[10])

31. What feature introduced in C99 allows array dimensions to be determined at runtime?
- **(A)** Variable-Length Arrays
- **(B)** Dynamic Arrays (malloc)
- **(C)** Flexible Struct Member
- **(D)** Template Arrays

32. Where are C99 Variable-Length Arrays (VLAs) allocated at runtime?
- **(A)** Call stack
- **(B)** Heap segment
- **(C)** Data segment
- **(D)** BSS segment

33. Can a C99 Variable-Length Array (VLA) have static storage duration?
- **(A)** No (stack-only)
- **(B)** Yes
- **(C)** Only in C11
- **(D)** Implementation-defined

34. What C99 feature allows initializing specific indices like: int a[10] = {[3] = 42, [7] = 99};?
- **(A)** Designated initializers
- **(B)** Index mapping
- **(C)** Compound literals
- **(D)** Static slicing

35. What does expression sizeof(arr) / sizeof(arr[0]) compute for a statically declared array?
- **(A)** Number of elements
- **(B)** Total bytes of array
- **(C)** Size of one element
- **(D)** Pointer offset

36. Why does sizeof(arr) / sizeof(arr[0]) fail inside a function receiving the array as a parameter?
- **(A)** Array decayed to pointer
- **(B)** sizeof invalid on param
- **(C)** Divides by zero
- **(D)** Array is truncated

37. What is a matrix with identical numbers of rows and columns called?
- **(A)** Square matrix
- **(B)** Rectangular matrix
- **(C)** Scalar matrix
- **(D)** Diagonal vector

38. In a square matrix M of dimension N x N, which elements satisfy i == j?
- **(A)** Main diagonal elements
- **(B)** Anti-diagonal elements
- **(C)** Upper triangle elements
- **(D)** Lower triangle elements

39. What matrix operation swaps rows and columns such that A_T[i][j] = A[j][i]?
- **(A)** Matrix Transpose
- **(B)** Matrix Inversion
- **(C)** Matrix Determinant
- **(D)** Matrix Trace

40. What is the condition for two matrices of dimensions (M x K) and (P x N) to be multiplicable?
- **(A)** K == P
- **(B)** M == N
- **(C)** M == K
- **(D)** K == N

41. What is the dimension of the product matrix resulting from (4 x 3) multiplied by (3 x 5)?
- **(A)** 4 x 5
- **(B)** 3 x 3
- **(C)** 5 x 4
- **(D)** 12 x 15

42. How many scalar multiplications are performed when multiplying two N x N matrices naively?
- **(A)** N^3
- **(B)** N^2
- **(C)** N * log N
- **(D)** 2 * N^2

43. What is the time complexity of naive matrix multiplication algorithm?
- **(A)** O(N^3)
- **(B)** O(N^2)
- **(C)** O(N log N)
- **(D)** O(2^N)

44. What is the sum of main diagonal elements of a square matrix termed in linear algebra?
- **(A)** Trace of matrix
- **(B)** Determinant
- **(C)** Rank
- **(D)** Eigenvalue

45. A square matrix where A[i][j] == A[j][i] for all i, j is defined as:
- **(A)** Symmetric matrix
- **(B)** Skew-symmetric matrix
- **(C)** Orthogonal matrix
- **(D)** Singular matrix

46. A square matrix where A[i][j] == -A[j][i] and main diagonal elements are zero is:
- **(A)** Skew-symmetric matrix
- **(B)** Symmetric matrix
- **(C)** Identity matrix
- **(D)** Hermitian matrix

47. What is an identity matrix?
- **(A)** 1s on diagonal, 0s elsewhere
- **(B)** All elements 1
- **(C)** All elements 0
- **(D)** Alternating 0 and 1

48. In Binary Search, what formula avoids integer overflow when computing the midpoint?
- **(A)** low + (high - low) / 2
- **(B)** (low + high) / 2
- **(C)** low + high / 2
- **(D)** high - low / 2

49. Why does (low + high) / 2 risk overflow in large arrays in C?
- **(A)** Sum exceeds INT_MAX
- **(B)** Division by zero
- **(C)** Negative rounding
- **(D)** Floating conversion

50. What optimization allows Bubble Sort to terminate in O(N) time on already sorted arrays?
- **(A)** Swapped boolean flag
- **(B)** Binary search midpoint
- **(C)** Recursive partitioning
- **(D)** Bitwise masking

51. Is standard Selection Sort a stable sorting algorithm?
- **(A)** No (unstable)
- **(B)** Yes (strictly stable)
- **(C)** Depends on pivot
- **(D)** Only for integers

52. What does it mean for a sorting algorithm to be 'stable'?
- **(A)** Preserves order of equals
- **(B)** Uses O(1) memory
- **(C)** Runs in O(N log N)
- **(D)** Never crashes

53. Which of the elementary O(N^2) sorting algorithms is stable by default?
- **(A)** Insertion Sort & Bubble
- **(B)** Selection Sort only
- **(C)** Quick Sort
- **(D)** Heap Sort

54. What is the maximum number of comparisons in worst-case Binary Search for N = 1024 elements?
- **(A)** 11 (log2(1024) + 1)
- **(B)** 1024
- **(C)** 512
- **(D)** 100

55. How many comparisons does Linear Search make in the worst case for N = 1000?
- **(A)** 1000
- **(B)** 10
- **(C)** 500
- **(D)** 1

56. What is the result of expression: int arr[5] = {10, 20, 30}; *(arr + 2);?
- **(A)** 30
- **(B)** 20
- **(C)** 10
- **(D)** Address of arr

57. In C, which relationship between array indexing and pointer arithmetic is universally true?
- **(A)** arr[i] == *(arr + i)
- **(B)** arr[i] == &arr + i
- **(C)** arr[i] == *arr + i
- **(D)** arr[i] == arr + *i

58. Is the expression 2[arr] syntactically legal and identical to arr[2] in ANSI C?
- **(A)** Yes: *(2 + arr)
- **(B)** No (syntax error)
- **(C)** Only in C++
- **(D)** Deprecated in C99

59. Why does 2[arr] work identically to arr[2] in C?
- **(A)** Addition is commutative
- **(B)** Compiler special case
- **(C)** Array indexing macro
- **(D)** Undefined behavior

60. What does the expression &arr represent for int arr[10]?
- **(A)** Pointer to array of 10 ints
- **(B)** Pointer to first int
- **(C)** Value of first int
- **(D)** Array size in bytes

61. What is the difference in pointer arithmetic between arr + 1 and &arr + 1?
- **(A)** &arr+1 skips ENTIRE array
- **(B)** Identical offset
- **(C)** arr+1 skips 10 ints
- **(D)** Causes syntax error

62. If int arr[10] starts at address 1000, what is address evaluated by &arr + 1 (int=4 bytes)?
- **(A)** 1040 (skips 10 * 4 bytes)
- **(B)** 1004
- **(C)** 1010
- **(D)** 1000

63. In bioinformatics, what does a 2D Dot Plot comparison matrix between two sequences visualize?
- **(A)** Sequence homology & repeats
- **(B)** Molecular weights
- **(C)** GC percentage curves
- **(D)** Hydrogen bond counts

64. In DNA Dot Plot matrix M[i][j], what value is assigned if seq1[i] == seq2[j]?
- **(A)** 1 (or dot symbol)
- **(B)** 0
- **(C)** -1
- **(D)** Index offset

65. What graphical pattern on a Dot Plot indicates an exact matching sequence alignment?
- **(A)** Diagonal line segment
- **(B)** Vertical column
- **(C)** Horizontal bar
- **(D)** Random checkerboard

66. What pattern on a Dot Plot indicates an inverted repeat (palindromic sequence)?
- **(A)** Anti-diagonal line
- **(B)** Vertical line
- **(C)** Horizontal gap
- **(D)** Scattered noise

67. In a gene expression matrix, rows represent genes and columns represent experimental samples:
- **(A)** Microarray Data Matrix
- **(B)** Phylogenetic Cladogram
- **(C)** Codon Dictionary
- **(D)** Restriction Map

68. What algorithm removes duplicates from a sorted array in O(N) time?
- **(A)** Two-pointer technique
- **(B)** Triple nested loops
- **(C)** Binary search insertion
- **(D)** Hash table recreation

69. How many element comparisons does standard Selection Sort make on an array of size N?
- **(A)** N * (N - 1) / 2
- **(B)** N
- **(C)** N * log N
- **(D)** N^2 - 1

70. What is the auxiliary space complexity of In-Place Bubble, Selection, and Insertion sorts?
- **(A)** O(1) constant space
- **(B)** O(N) linear space
- **(C)** O(log N)
- **(D)** O(N^2)

71. What is a major performance drawback of Insertion Sort on large reverse-sorted arrays?
- **(A)** O(N^2) shifts and moves
- **(B)** High recursion overhead
- **(C)** Large memory allocation
- **(D)** Cache misses only

72. Can an array dimension in C89 be declared using a const variable: const int n = 5; int a[n];?
- **(A)** No (const var not const expr)
- **(B)** Yes (valid in C89)
- **(C)** Only if global
- **(D)** Only if static

73. Why was const int n = 5; int a[n]; illegal in ANSI C89?
- **(A)** const is read-only variable
- **(B)** C89 had no const keyword
- **(C)** Arrays must be dynamic
- **(D)** n had no memory address

74. How did C99 resolve dynamic stack array sizing?
- **(A)** Introduced VLAs
- **(B)** Removed static arrays
- **(C)** Changed const semantics
- **(D)** Added new malloc syntax

75. In C11, what change was made regarding Variable-Length Arrays (VLAs)?
- **(A)** Made optional feature
- **(B)** Removed from standard
- **(C)** Made mandatory everywhere
- **(D)** Restricted to char arrays

76. What is the optimal algorithm to rotate an N x N matrix by 90 degrees clockwise in-place?
- **(A)** Transpose then reverse rows
- **(B)** Reverse rows then transpose
- **(C)** Rotate four corners iteratively
- **(D)** Both A and C are valid

77. In rotating a matrix 90 degrees clockwise: Transpose matrix, then reverse each row. What is complexity?
- **(A)** O(N^2) time, O(1) space
- **(B)** O(N^3) time
- **(C)** O(N^2) space
- **(D)** O(N) time

78. What is the optimal in-place algorithm to rotate an array of size N by K positions to the right?
- **(A)** Triple reverse algorithm
- **(B)** Shift elements one-by-one
- **(C)** Allocate secondary array
- **(D)** Bubble sort rotation

79. In the Triple Reverse algorithm to rotate array right by K: what are the 3 steps?
- **(A)** Reverse all, rev 0..K-1, rev K..N-1
- **(B)** Rev 0..K, rev K+1..N, rev all
- **(C)** Shift K times
- **(D)** Swap pairs

80. What is the time and auxiliary space complexity of the Triple Reverse array rotation?
- **(A)** O(N) time, O(1) space
- **(B)** O(N*K) time, O(1) space
- **(C)** O(N) space
- **(D)** O(log N) time

81. What algorithm finds the contiguous subarray within a 1D array having the largest sum in O(N)?
- **(A)** Kadane's Algorithm
- **(B)** Dijkstra's Algorithm
- **(C)** Boyer-Moore Voting
- **(D)** Floyd's Cycle Algorithm

82. In Kadane's algorithm: max_ending_here = max(arr[i], max_ending_here + arr[i]), what is time complexity?
- **(A)** O(N) linear time
- **(B)** O(N^2)
- **(C)** O(N log N)
- **(D)** O(1)

83. What algorithm finds the majority element (occurring > N/2 times) in an array in O(N) time and O(1) space?
- **(A)** Boyer-Moore Voting Algorithm
- **(B)** Binary Search
- **(C)** Quickselect
- **(D)** Counting Sort

84. In matrix multiplication C[i][j] += A[i][k] * B[k][j], which loop ordering optimizes CPU cache locality?
- **(A)** i-k-j loop ordering
- **(B)** i-j-k standard ordering
- **(C)** j-i-k ordering
- **(D)** k-j-i ordering

85. Why is i-k-j loop ordering faster than standard i-j-k for large matrix multiplication in C?
(A) B is accessed row-wise in stride-1
(B) Eliminates inner loop
(C) Uses less RAM
(D) Inlines automatically

86. In bioinformatics, what dynamic programming scoring matrix algorithm performs global DNA sequence alignment?
(A) Needleman-Wunsch Algorithm
(B) Smith-Waterman Algorithm
(C) BLAST Algorithm
(D) Viterbi Algorithm

87. What dynamic programming algorithm performs local sequence alignment in bioinformatics?
(A) Smith-Waterman Algorithm
(B) Needleman-Wunsch Algorithm
(C) KMP Algorithm
(D) Aho-Corasick

88. What are the dimensions of the Needleman-Wunsch DP alignment matrix for sequences of length M and N?
(A) (M + 1) x (N + 1)
(B) M x N
(C) (M - 1) x (N - 1)
(D) 2 x (M + N)

89. In Needleman-Wunsch matrix filling, cell F(i, j) takes the maximum of how many possible transitions?
(A) 3 (Match/Mismatch, Del, Ins)
(B) 2
(C) 4
(D) 1

90. What is the time and space complexity of standard Needleman-Wunsch alignment for sequences of length N?
(A) O(N^2) time and space
(B) O(N) time
(C) O(N log N) time
(D) O(2^N) time

91. How does Hirschberg's algorithm optimize Needleman-Wunsch global alignment space complexity?
(A) Reduces space to O(N)
(B) Reduces time to O(N)
(C) Eliminates gap penalties
(D) Sorts sequences first

92. In an array representing a binary min-heap, for a node at index i (0-based), what is the left child index?
(A) 2*i + 1
(B) 2*i
(C) 2*i + 2
(D) i / 2

93. For a node at index i in a 0-based binary heap, what is the right child index?
(A) 2*i + 2
(B) 2*i + 1
(C) 2*i
(D) (i - 1) / 2

94. For a node at index i in a 0-based binary heap, what is the parent node index?
(A) (i - 1) / 2
(B) i / 2
(C) (i + 1) / 2
(D) 2 * i

95. What algorithm sorts an array of integers with small key range [0..K] in O(N + K) linear time?
(A) Counting Sort
(B) Bubble Sort
(C) Merge Sort
(D) Quick Sort

96. Is Counting Sort a comparison-based sorting algorithm?
(A) No (non-comparison based)
(B) Yes (compares pairs)
(C) Only for floats
(D) Implementation defined

97. What is the theoretical lower bound on time complexity for any comparison-based sorting algorithm?
(A) Omega(N log N)
(B) Omega(N)
(C) Omega(N^2)
(D) Omega(1)

98. Why cannot any comparison-based sorting algorithm beat O(N log N) in the worst case?
(A) Decision tree height log2(N!)
(B) Memory bandwidth limits
(C) Register spill latency
(D) Cache line boundaries

99. What happens when an expression attempts to read arr[-1] in standard ANSI C?

(A) Undefined behavior (illegal read)
(B) Returns 0
(C) Returns last element
(D) Compilation error

100. Can a pointer be offset such that ptr[-1] is valid legal code in C?

(A) Yes: if ptr = arr + 1
(B) No (strictly forbidden)
(C) Only in GCC extensions
(D) Only for char arrays

SECTION B: DESCRIPTIVE & SUBJECTIVE QUESTIONS

Q1. Explain Array Fundamentals in C, detailing contiguous memory allocation, zero-based indexing, and address calculation formulas.

Q2. Detail array initialization techniques in ANSI C, contrasting full initialization, partial initialization, and C99 designated initializers.

Q3. Analyze the absence of Array Bounds Checking in C, explaining buffer overflow vulnerabilities and defensive bounds validation.

Q4. Explain Multi-Dimensional Arrays in C, detailing 2D row-major layout, memory offset formulas, and matrix transformations.

Q5. Write an ANSI C program to perform Matrix Multiplication C = A x B, explaining dimensional constraints and cache locality.

Q6. Write an ANSI C program to compute the Transpose of an N x N square matrix in-place without allocating auxiliary memory.

Q7. Write an ANSI C program to implement Linear Search and Binary Search on an array, analyzing comparative search benchmarks.

Q8. Write an ANSI C program to implement Bubble Sort with an early-termination swap flag optimization, analyzing best and worst cases.

Q9. Write an ANSI C program to implement Selection Sort, explaining why it performs exactly N(N-1)/2 comparisons and its instability.

Q10. Write an ANSI C program to implement Insertion Sort, explaining its adaptive nature on nearly sorted arrays and stability.

Q11. Write an ANSI C program to calculate mean, variance, and standard deviation of a floating-point experimental dataset array.

Q12. Write an ANSI C program to construct a DNA Sequence Dot Plot comparison matrix between two biological sequences.

Q13. Write an ANSI C program to remove duplicate elements from an array in-place, comparing unsorted vs sorted approaches.

Q14. Write an ANSI C program to verify whether a square matrix is Symmetric, Skew-Symmetric, or Asymmetric.

Q15. Explain Kadane's Algorithm for finding the maximum contiguous subarray sum in O(N) time with code and tracing.

Q16. Write an ANSI C program to rotate an array of size N by K positions to the right using the Triple Reverse Algorithm in O(N) time and O(1) space.

Q17. Explain the Boyer-Moore Majority Voting Algorithm in C, demonstrating O(N) time and O(1) space majority detection.

Q18. Write an ANSI C program to implement Counting Sort on an array of integers within a fixed range, analyzing linear time complexity.

Q19. Explain the Needleman-Wunsch algorithm for global DNA sequence alignment using a 2D dynamic programming array.

Q20. Write an ANSI C program to compute the Spiral Order Traversal of an M x N 2D matrix.

Q21. Explain Variable-Length Arrays (VLAs) in C99, detailing stack allocation, sizeof evaluation, and security hazards.

Q22. Write an ANSI C program to rotate an N x N square matrix 90 degrees clockwise in-place.

Q23. Write an ANSI C program to merge two pre-sorted integer arrays into a single sorted array in O(N + M) time.

Q24. Explain passing multidimensional arrays to functions with variable dimensions using C99 Variable-Length Array parameters.

Q25. Write an ANSI C program to find the second largest element in an array in a single traversal pass in O(N) time.

Q26. Write an ANSI C program to check whether a given square matrix is an Identity Matrix.

Q27. Explain the two-pointer approach for finding a pair of elements in a sorted array that sum to a target value S.

Q28. Write an ANSI C program to compute the trace (sum of main diagonal elements) of a square matrix.

Q29. Explain cache miss penalties during row-major vs column-major array traversals in C with concrete timing considerations.

Q30. Write an ANSI C program to sort biological DNA fragments by molecular length using Insertion Sort.

SECTION C: ANSWER KEY - MULTIPLE CHOICE QUESTIONS

Q1:	[A]	**Q21:**	[A]	**Q41:**	[A]	**Q61:**	[A]	**Q81:**	[A]
Q2:	[A]	**Q22:**	[A]	**Q42:**	[A]	**Q62:**	[A]	**Q82:**	[A]
Q3:	[A]	**Q23:**	[A]	**Q43:**	[A]	**Q63:**	[A]	**Q83:**	[A]
Q4:	[A]	**Q24:**	[A]	**Q44:**	[A]	**Q64:**	[A]	**Q84:**	[A]
Q5:	[A]	**Q25:**	[A]	**Q45:**	[A]	**Q65:**	[A]	**Q85:**	[A]
Q6:	[A]	**Q26:**	[A]	**Q46:**	[A]	**Q66:**	[A]	**Q86:**	[A]
Q7:	[A]	**Q27:**	[A]	**Q47:**	[A]	**Q67:**	[A]	**Q87:**	[A]
Q8:	[A]	**Q28:**	[A]	**Q48:**	[A]	**Q68:**	[A]	**Q88:**	[A]
Q9:	[A]	**Q29:**	[A]	**Q49:**	[A]	**Q69:**	[A]	**Q89:**	[A]
Q10:	[A]	**Q30:**	[A]	**Q50:**	[A]	**Q70:**	[A]	**Q90:**	[A]
Q11:	[A]	**Q31:**	[A]	**Q51:**	[A]	**Q71:**	[A]	**Q91:**	[A]
Q12:	[A]	**Q32:**	[A]	**Q52:**	[A]	**Q72:**	[A]	**Q92:**	[A]
Q13:	[A]	**Q33:**	[A]	**Q53:**	[A]	**Q73:**	[A]	**Q93:**	[A]
Q14:	[A]	**Q34:**	[A]	**Q54:**	[A]	**Q74:**	[A]	**Q94:**	[A]
Q15:	[A]	**Q35:**	[A]	**Q55:**	[A]	**Q75:**	[A]	**Q95:**	[A]
Q16:	[A]	**Q36:**	[A]	**Q56:**	[A]	**Q76:**	[D]	**Q96:**	[A]
Q17:	[A]	**Q37:**	[A]	**Q57:**	[A]	**Q77:**	[A]	**Q97:**	[A]
Q18:	[A]	**Q38:**	[A]	**Q58:**	[A]	**Q78:**	[A]	**Q98:**	[A]
Q19:	[A]	**Q39:**	[A]	**Q59:**	[A]	**Q79:**	[A]	**Q99:**	[A]
Q20:	[A]	**Q40:**	[A]	**Q60:**	[A]	**Q80:**	[A]	**Q100:**	[A]

MCQ Scoring Guide & Difficulty Distribution:

Questions 1 to 40: Foundational & Conceptual [Easy] | Questions 41 to 75: Analytical & Applied [Moderate]

Questions 76 to 100: Complex Numerical & Advanced Problem Solving [Difficult]

SECTION B: DETAILED SOLUTIONS FOR SUBJECTIVE QUESTIONS

1. Explain the concept of contiguous memory allocation, zero-based indexing, and address calculation formulas.

[Problem Statement]: Detail array initialization techniques in ANSI C, contrasting full initialization, partial initialization, and C99 designated initializ
[Detailed Step-by-Step Answer]:
Array Initialization Techniques in ANSI C:
1. Full Static Initialization:
- Elements are enclosed within curly braces separated by commas:
int prime_factors[5] = {2, 3, 5, 7, 11};
- Array Size Inference: If size dimension is omitted, compiler computes size from initializer
count:
int primes[] = {2, 3, 5, 7, 11}; /* Automatically sized to 5 elements */

2. Partial Initialization and Zero-Fill Guarantee:
- If fewer initializers are supplied than the declared capacity:
int buffer[10] = {1, 2, 3};
- ISO C Guarantee: The remaining unspecified elements are AUTOMATICALLY INITIALIZED TO ZERO!
- Idiomatic Zero-Clear:
int zero_array[100] = {0}; /* Guaranteed all 100 elements are zeroed */

3. Uninitialized Automatic Arrays Trap:
- Declaring int data[50]; inside a function without an initializer leaves memory uninitialized.
- Reading data[i] retrieves indeterminate garbage bits leftover from previous stack frames,
causing non-deterministic application bugs.

4. C99 Designated Initializers:
- Allows explicit initialization of designated array indices in any arbitrary order:
int lookup_table[10] = { [2] = 200, [7] = 700, [4] = 400 };
- Any indices not explicitly designated are automatically zeroed.

5. Demonstrative ANSI C Program:

```
#include <stdio.h>

int main(void) {
/* 1. Partial initialization with zero-fill */
int partial_arr[6] = {10, 20};
printf("Partial array: ");
for (int i = 0; i < 6; i++) printf("%d ", partial_arr[i]);
printf("\n");

/* 2. C99 Designated initializer */
int codon_weights[8] = {[0] = 15, [4] = 95, [7] = 120};
printf("Designated array: ");
for (int i = 0; i < 8; i++) printf("[%d]:%d ", i, codon_weights[i]);
printf("\n");

return 0;
}
```

>>[Solution to Question 3: Analyze the absence of Array Bounds Checking in C, explaining buffer overflow vulnerabilities and defensive bounds validation.

ArrayIndexOutOfBoundsException.
- In ANSI C, THERE IS ZERO RUNTIME ARRAY BOUNDS CHECKING!
- Philosophy: C prioritizes maximum hardware execution velocity. Adding runtime boundary checks on every pointer arithmetic instruction incurs heavy CPU branch overhead.

2. Out-of-Bounds Memory Corruption:
- When code accesses arr[15] on an array of size 10:
Calculated address: Base + 15 * sizeof(int).
The CPU reads or writes directly to whatever memory resides at that offset!
- Stack Buffer Overflow: Overwriting stack memory corrupts adjacent local variables, stack canaries, saved frame pointers (EBP/RBP), and the function's Return Address.
- Exploitation: Attackers supply oversized inputs that overwrite the return address with the address of malicious shellcode, hijacking control of the process.

3. Complete Demonstration Code:

```
#include <stdio.h>

void vulnerable_function(void) {
int target_variable = 0;
int array_buffer[4] = {10, 20, 30, 40};

printf("Target variable BEFORE overflow: %d\n", target_variable);

/* Out-of-bounds write: Writing beyond index 3 */
/* On typical stack layouts, index 4 or 5 aliases target_variable! */
array_buffer[4] = 9999; /* UNDEFINED BEHAVIOR! */

printf("Target variable AFTER overflow: %d (CORRUPTED!)\n", target_variable);
}

int main(void) {
vulnerable_function();
return 0;
}
```

4. Defensive Standards:
- Always validate indices: if (index >= 0 && index < ARRAY_SIZE).
- Use compile-time warnings: gcc -Wall -Wextra -Warray-bounds -fsanitize=address.

>> Solution to Question 54: Explain Multi-Dimensional Arrays in C, detailing 2D row-major layout, memory offset formulas, and matrix transformations.

followed by Row 2, etc.
- Contrast with Fortran and MATLAB, which use Column-Major Order.

3. 2D Memory Address Formula:
Address(M[i][j]) = BaseAddress + (i * COLS + j) * sizeof(ElementType)
- i * COLS: Skips 'i' complete rows of width COLS.
- + j: Offsets to the j-th element in the target row.
- Notice: The address calculation depends ONLY on the number of COLUMNS, not on the total number of rows! This is why function declarations MUST include the column dimension!

4. Demonstrative ANSI C Program:

```
#include <stdio.h>

#define ROWS 3
#define COLS 4

int main(void) {
int matrix[ROWS][COLS] = {
{11, 12, 13, 14},
{21, 22, 23, 24},
{31, 32, 33, 34}
};

printf("=== 2D Row-Major Memory Mapping ===\n");
for (int i = 0; i < ROWS; i++) {
for (int j = 0; j < COLS; j++) {
printf("M[%d][%d] = %2d at %p\n", i, j, matrix[i][j], (void *)&matrix[i][j]);
}
}

/* Demonstrate linear equivalence via single pointer */
int *linear_ptr = &matrix[0][0];
printf("\nAccessing M[2][1] linearly (offset 2*4 + 1 = 9): %d\n", linear_ptr[2 * COLS + 1]);

return 0;
}
```

>>Solution to Question 5: Here is an ANSI C program to perform Matrix Multiplication C = A x B, explaining dimensional constraints and cache locality.

3. Cache Locality Consideration:
In the standard loop order (i, j, k), elements of B[k][j] are accessed with a stride of COLS_B (column-wise), causing frequent CPU cache line misses. Reordering loops to (i, k, j) accesses both A and B with stride-1 (row-wise), accelerating execution by 3x-5x on modern CPUs!

4. Complete ANSI C Source Code:

```
#include <stdio.h>

#define M 2
#define K 3
#define N 2

void multiply_matrices(const int A[M][K], const int B[K][N], int C[M][N]) {
/* Initialize product matrix C to zero */
for (int i = 0; i < M; i++) {
for (int j = 0; j < N; j++) {
C[i][j] = 0;
}
}

/* Cache-friendly i-k-j loop ordering */
for (int i = 0; i < M; i++) {
for (int k = 0; k < K; k++) {
int r = A[i][k];
for (int j = 0; j < N; j++) {
C[i][j] += r * B[k][j];
}
}
}
}

int main(void) {
int A[M][K] = {{1, 2, 3}, {4, 5, 6}};
int B[K][N] = {{7, 8}, {9, 1}, {2, 3}};
int C[M][N];

multiply_matrices(A, B, C);

printf("=== Resulting Product Matrix (2x2) ===\n");
for (int i = 0; i < M; i++) {
for (int j = 0; j < N; j++) {
printf("%6d ", C[i][j]);
}
printf("\n");
}
return 0;
}
```

>> Solution to Question 6: Here is an ANSI C program to compute the Transpose of an N x N square matrix in-place without allocating auxiliary memory.

In-Place Matrix Transposition:
1. Mathematical Definition:
The transpose of matrix A, denoted A^T, swaps rows and columns:
A^T[i][j] = A[j][i].
- Main diagonal elements (where i == j) remain completely unchanged.
- Non-diagonal elements are symmetric reflections across the main diagonal.

2. In-Place Swapping Strategy:
- For an N x N square matrix, swapping can be performed in-place using O(1) auxiliary space.
- Traversal Boundary: Loop 'i' from 0 to N-1, and loop 'j' strictly from (i + 1) to N-1
(strictly upper triangle).
- Swap A[i][j] with A[j][i].
- Caution: If 'j' loops from 0 to N-1, elements are swapped twice, returning the matrix to its
original state!

3. Complete ANSI C Source Code:

```
#include <stdio.h>

#define N 4

void transpose_inplace(int matrix[N][N]) {
for (int i = 0; i < N; i++) {
for (int j = i + 1; j < N; j++) {
/* Swap element across main diagonal */
int temp = matrix[i][j];
matrix[i][j] = matrix[j][i];
matrix[j][i] = temp;
}
}
}

void print_square_matrix(const int matrix[N][N]) {
for (int i = 0; i < N; i++) {
for (int j = 0; j < N; j++) {
printf("%4d ", matrix[i][j]);
}
printf("\n");
}
}

int main(void) {
int M[N][N] = {
{ 1, 2, 3, 4},
{ 5, 6, 7, 8},
{ 9, 10, 11, 12},
{13, 14, 15, 16}
};

printf("Original 4x4 Matrix:\n");
print_square_matrix(M);

transpose_inplace(M);

printf("\nTransposed Matrix (In-Place):\n");
```

```
  print_square_matrix(M);

  return 0;
  }
```

>> Solution to Question 7: Write an ANSI C program to implement Linear Search and Binary Search on an array, analyzing comparative search benchmarks.

```
  int dataset[] = {12, 25, 38, 47, 59, 63, 71, 88, 92, 105, 140, 200};
  int n = sizeof(dataset) / sizeof(dataset[0]);
  int query = 92;
  int comp_lin = 0, comp_bin = 0;

  int idx_lin = linear_search(dataset, n, query, &comp_lin);
  int idx_bin = binary_search(dataset, n, query, &comp_bin);

  printf("=== Search Algorithm Benchmark for Target %d ===\n", query);
  printf("Linear Search: Index = %2d | Comparisons Made: %2d\n", idx_lin, comp_lin);
  printf("Binary Search: Index = %2d | Comparisons Made: %2d\n", idx_bin, comp_bin);

  return 0;
  }
```

>>Solution to Question 8: Here is an ANSI C program to implement Bubble Sort with an early-termination swap flag optimization, analyzing best and worst cases.

```
/* If no swaps occurred, terminate early */
if (!swapped) break;
}
}

void print_arr(const int *arr, int n) {
for (int i = 0; i < n; i++) printf("%d ", arr[i]);
printf("\n");
}

int main(void) {
int data[] = {64, 34, 25, 12, 22, 11, 90};
int n = sizeof(data) / sizeof(data[0]);
int passes = 0;

printf("Unsorted Array: ");
print_arr(data, n);

bubble_sort_optimized(data, n, &passes);

printf("Sorted Array: ");
print_arr(data, n);
printf("Total Passes Required: %d of %d maximum\n", passes, n - 1);

return 0;
}
```

>>Solution to Question 9: Here is an ANSI C program to implement Selection Sort, explaining why it performs exactly N(N-1)/2 comparisons and its instability.

4. Complete ANSI C Source Code:

```
#include <stdio.h>

void selection_sort(int *arr, int n) {
for (int i = 0; i < n - 1; i++) {
int min_idx = i;
for (int j = i + 1; j < n; j++) {
if (arr[j] < arr[min_idx]) {
min_idx = j;
}
}
if (min_idx != i) {
int temp = arr[i];
arr[i] = arr[min_idx];
arr[min_idx] = temp;
}
}
}

int main(void) {
int gene_sizes[] = {540, 120, 890, 320, 210, 750};
int n = sizeof(gene_sizes) / sizeof(gene_sizes[0]);

selection_sort(gene_sizes, n);

printf("Sorted Gene Sizes (bp): ");
for (int i = 0; i < n; i++) printf("%d ", gene_sizes[i]);
printf("\n");

return 0;
}
```

>> Solution to Question 10: Write an ANSI C program to implement Insertion Sort, explaining its adaptive nature on nearly sorted arrays and stability.

3. Complete ANSI C Source Code:

```
#include <stdio.h>

void insertion_sort(int *arr, int n) {
for (int i = 1; i < n; i++) {
int key = arr[i];
int j = i - 1;

/* Shift elements greater than key to the right */
while (j >= 0 && arr[j] > key) {
arr[j + 1] = arr[j];
j--;
}
arr[j + 1] = key; /* Place key into insertion slot */
}
}

int main(void) {
int fragment_lengths[] = {45, 12, 85, 32, 89, 39, 69, 22};
int n = sizeof(fragment_lengths) / sizeof(fragment_lengths[0]);

insertion_sort(fragment_lengths, n);

printf("Sorted Fragment Lengths: ");
for (int i = 0; i < n; i++) printf("%d ", fragment_lengths[i]);
printf("\n");

return 0;
}
```

>> Solution to Question 11: an ANSI C program to calculate mean, variance, and standard deviation of a floating-point experimental dataset array.

```
*mean = (n == 1) ? data[0] : 0.0;
*variance = 0.0;
*std_dev = 0.0;
return;
}

/* Pass 1: Compute Arithmetic Mean */
double sum = 0.0;
for (int i = 0; i < n; i++) {
sum += data[i];
}
*mean = sum / (double)n;

/* Pass 2: Compute Sum of Squared Differences */
double sum_sq_diff = 0.0;
for (int i = 0; i < n; i++) {
double diff = data[i] - *mean;
sum_sq_diff += (diff * diff);
}

*variance = sum_sq_diff / (double)(n - 1); /* Bessel's correction */
*std_dev = sqrt(*variance);
}

int main(void) {
double protein_concentrations[] = {12.4, 14.2, 11.8, 13.5, 12.9, 15.1, 13.8};
int n = sizeof(protein_concentrations) / sizeof(protein_concentrations[0]);

double m, v, s;
compute_dataset_statistics(protein_concentrations, n, &m, &v, &s);

printf("=== Experimental Protein Assay Statistics ===\n");
printf("Sample Size (N): %d\n", n);
printf("Arithmetic Mean: %8.4f ug/mL\n", m);
printf("Sample Variance: %8.4f\n", v);
printf("Standard Deviation (s): %8.4f ug/mL\n", s);

return 0;
}
```

>>Solution to Question 12: an ANSI C program to construct a DNA Sequence Dot Plot comparison matrix between two biological sequences.

2. Complete ANSI C Source Code:

```
#include <stdio.h>
#include <string.h>
#include <ctype.h>

#define MAX_SEQ 20

void generate_dot_plot(const char *seq1, const char *seq2) {
int len1 = strlen(seq1);
int len2 = strlen(seq2);

printf("=== 2D DNA Sequence Alignment Dot Plot ===\n\n ");
/* Print horizontal header */
for (int j = 0; j < len2; j++) {
printf(" %c", seq2[j]);
}
printf("\n +");
for (int j = 0; j < len2; j++) printf("--");
printf("\n");

/* Matrix row iteration */
for (int i = 0; i < len1; i++) {
printf(" %c |", seq1[i]);
for (int j = 0; j < len2; j++) {
if (toupper((unsigned char)seq1[i]) == toupper((unsigned char)seq2[j])) {
printf(" *");
} else {
printf(" .");
}
}
printf("\n");
}
}

int main(void) {
const char *geneA = "ATGCGATCG";
const char *geneB = "ATCCGATCG";

generate_dot_plot(geneA, geneB);
return 0;
}
```

>>Solution to Question 12: an ANSI C program to remove duplicate elements from an array in-place, comparing unsorted vs sorted approaches.

2. Sorted Array Two-Pointer Approach:
- Sort array first (or assume sorted input).
- Two Pointers: 'write_idx' tracks the boundary of unique elements; 'read_idx' traverses the array.
- Since identical elements are grouped together, simply check if arr[read_idx] != arr[write_idx]:
If true: increment write_idx and copy arr[read_idx] to arr[write_idx].
- Time Complexity: O(N) linear time; O(1) auxiliary space!

3. Complete ANSI C Source Code:

```
#include <stdio.h>

int remove_duplicates_sorted(int *arr, int n) {
if (n <= 1) return n;

int write_idx = 0;
for (int read_idx = 1; read_idx < n; read_idx++) {
if (arr[read_idx] != arr[write_idx]) {
write_idx++;
arr[write_idx] = arr[read_idx];
}
}
return write_idx + 1; /* Return length of unique prefix */
}

int main(void) {
int sorted_codons[] = {10, 10, 20, 30, 30, 30, 40, 50, 50, 60};
int n = sizeof(sorted_codons) / sizeof(sorted_codons[0]);

printf("Original Sorted Array (%d elements):\n", n);
for (int i = 0; i < n; i++) printf("%d ", sorted_codons[i]);
printf("\n");

int unique_count = remove_duplicates_sorted(sorted_codons, n);

printf("Array after in-place deduplication (%d unique elements):\n", unique_count);
for (int i = 0; i < unique_count; i++) printf("%d ", sorted_codons[i]);
printf("\n");

return 0;
}
```

>>Solution to Question 14: an ANSI C program to verify whether a square matrix is Symmetric, Skew-Symmetric, or Asymmetric.

2. Algorithmic Optimization:
Only need to inspect elements where j > i (strictly upper triangle) and verify main diagonal zeros for skew-symmetry.

3. Complete ANSI C Source Code:

```
#include <stdio.h>
#include <stdbool.h>

#define N 3

void classify_matrix_symmetry(const int M[N][N]) {
bool is_sym = true;
bool is_skew = true;

/* Check main diagonal zeros for skew symmetry */
for (int i = 0; i < N; i++) {
if (M[i][i] != 0) {
is_skew = false;
break;
}
}

/* Check non-diagonal reflections */
for (int i = 0; i < N; i++) {
for (int j = i + 1; j < N; j++) {
if (M[i][j] != M[j][i]) {
is_sym = false;
}
if (M[i][j] != -M[j][i]) {
is_skew = false;
}
}
}

if (is_sym) {
printf("Matrix is SYMMETRIC (M = M^T)\n");
} else if (is_skew) {
printf("Matrix is SKEW-SYMMETRIC (M = -M^T)\n");
} else {
printf("Matrix is ASYMMETRIC\n");
}
}

int main(void) {
int sym_mat[N][N] = {{1, 7, 3}, {7, 4, -5}, {3, -5, 6}};
int skew_mat[N][N] = {{0, 2, -5}, {-2, 0, 4}, {5, -4, 0}};

printf("Testing Matrix 1:\n");
classify_matrix_symmetry(sym_mat);

printf("Testing Matrix 2:\n");
classify_matrix_symmetry(skew_mat);
```

```
return 0;
}
```

>>Solution to Question 15: Explain Kadane's Algorithm for finding the maximum contiguous subarray sum in O(N) time with code and tracing.

```
  int max_sum = kadane_max_subarray(hydro_scores, n, &start, &end);

  printf("=== Maximum Subarray (Hydrophobicity Cluster) ===\n");
  printf("Maximum Contiguous Sum: %d\n", max_sum);
  printf("Subarray Range: indices [%d to %d]\n", start, end);
  printf("Elements: ");
  for (int i = start; i <= end; i++) printf("%d ", hydro_scores[i]);
  printf("\n");

  return 0;
  }
```

>>P Solution Statement for W16: an ANSI C program to rotate an array of size N by K positions to the right using the Triple Reverse Algorithm in O(N) time and O(

```
/* 2. Reverse first k elements */
reverse_subarr(arr, 0, k - 1);
/* 3. Reverse remaining n-k elements */
reverse_subarr(arr, k, n - 1);
}

int main(void) {
int arr[] = {1, 2, 3, 4, 5, 6, 7};
int n = sizeof(arr) / sizeof(arr[0]);
int k = 3;

rotate_array_right(arr, n, k);

printf("Rotated Array (right by %d): ", k);
for (int i = 0; i < n; i++) printf("%d ", arr[i]);
printf("\n");

return 0;
}
```

>> Solution to Question 17: Explain the Boyer-Moore Majority Voting Algorithm in C, demonstrating O(N) time and O(1) space majority detection.

```
  count--;
  }
  }

  /* Pass 2: Verify candidate */
  int occurrences = 0;
  for (int i = 0; i < n; i++) {
  if (arr[i] == candidate) occurrences++;
  }

  if (occurrences > n / 2) return candidate;
  return -1; /* No majority element exists */
  }

  int main(void) {
  int sample_cluster[] = {2, 2, 1, 1, 2, 2, 2};
  int n = sizeof(sample_cluster) / sizeof(sample_cluster[0]);

  int maj = find_majority_element(sample_cluster, n);
  if (maj != -1) {
  printf("Dominant Genetic Cluster: %d (verified > %d elements)\n", maj, n / 2);
  } else {
  printf("No majority cluster detected.\n");
  }
  return 0;
  }
```

>>P Solution to Question 18: an ANSI C program to implement Counting Sort on an array of integers within a fixed range, analyzing linear time complexity,

```
}

/* 2. Cumulative prefix sums */
for (int i = 1; i <= max_val; i++) {
count[i] += count[i - 1];
}

/* 3. Build output array stably (backwards) */
for (int i = n - 1; i >= 0; i--) {
output[count[arr[i]] - 1] = arr[i];
count[arr[i]]--;
}

/* 4. Copy back to original array */
memcpy(arr, output, n * sizeof(int));

free(output);
free(count);
}

int main(void) {
int phred_quality_scores[] = {34, 12, 38, 25, 34, 12, 40, 28, 30};
int n = sizeof(phred_quality_scores) / sizeof(phred_quality_scores[0]);

counting_sort(phred_quality_scores, n, 42);

printf("Sorted Phred Quality Scores: ");
for (int i = 0; i < n; i++) printf("%d ", phred_quality_scores[i]);
printf("\n");

return 0;
}
```

Solution to Question 19:

>> Explain the Needleman-Wunsch algorithm for global DNA sequence alignment using a 2D dynamic programming array.

F[i][j - 1] + GapPenalty /* Left: Insertion in B (Gap in A) */
)

3. Traceback Phase:
Begins at cell F[M][N] (bottom-right) and traces backward to F[0][0], reconstructing aligned sequences and gaps ('-').
Complexity: O(M * N) time and space.

4. Core ANSI C Implementation Listing:

```
#include <stdio.h>

#define MATCH_SCORE 1
#define MISMATCH_SCORE -1
#define GAP_PENALTY -2

int max3(int a, int b, int c) {
int m = a;
if (b > m) m = b;
if (c > m) m = c;
return m;
}

void needleman_wunsch_score(const char *s1, int len1, const char *s2, int len2) {
int dp[10][10];

/* Initialize boundaries */
for (int i = 0; i <= len1; i++) dp[i][0] = i * GAP_PENALTY;
for (int j = 0; j <= len2; j++) dp[0][j] = j * GAP_PENALTY;

/* Fill DP matrix */
for (int i = 1; i <= len1; i++) {
for (int j = 1; j <= len2; j++) {
int score_diag = dp[i-1][j-1] + ((s1[i-1] == s2[j-1]) ? MATCH_SCORE : MISMATCH_SCORE);
int score_up = dp[i-1][j] + GAP_PENALTY;
int score_left = dp[i][j-1] + GAP_PENALTY;
dp[i][j] = max3(score_diag, score_up, score_left);
}
}

printf("Optimal Global Alignment Score: %d\n", dp[len1][len2]);
}

int main(void) {
needleman_wunsch_score("GATT", 4, "GCAT", 4);
return 0;
}
```

>>Solution to Question 20: Write an ANSI C program to compute the Spiral Order Traversal of an M x N 2D matrix.

Given an M x N matrix, traverse all elements in clockwise spiral order:
Maintain four boundaries: top = 0, bottom = M - 1, left = 0, right = N - 1.
Loop while (top <= bottom && left <= right):
- Direction 1: Traverse from left to right along row 'top', then top++.
- Direction 2: Traverse from top to bottom along col 'right', then right--.
- Direction 3: If top <= bottom: Traverse right to left along row 'bottom', then bottom--.
- Direction 4: If left <= right: Traverse bottom to top along col 'left', then left++.

2. Complete ANSI C Source Code:

```
#include <stdio.h>

#define ROWS 3
#define COLS 4

void print_spiral(const int mat[ROWS][COLS], int m, int n) {
int top = 0, bottom = m - 1;
int left = 0, right = n - 1;

printf("Spiral Traversal: ");
while (top <= bottom && left <= right) {
/* Traverse right */
for (int j = left; j <= right; j++) printf("%d ", mat[top][j]);
top++;

/* Traverse down */
for (int i = top; i <= bottom; i++) printf("%d ", mat[i][right]);
right--;

/* Traverse left */
if (top <= bottom) {
for (int j = right; j >= left; j--) printf("%d ", mat[bottom][j]);
bottom--;
}

/* Traverse up */
if (left <= right) {
for (int i = bottom; i >= top; i--) printf("%d ", mat[i][left]);
left++;
}
}
printf("\n");
}

int main(void) {
int grid[ROWS][COLS] = {
{ 1, 2, 3, 4},
{ 5, 6, 7, 8},
{ 9, 10, 11, 12}
};
print_spiral(grid, ROWS, COLS);
return 0;
}
```

```
/* Step 2: Reverse each row */
for (int i = 0; i < N; i++) {
for (int j = 0; j < N / 2; j++) {
int temp = mat[i][j];
mat[i][j] = mat[i][N - 1 - j];
mat[i][N - 1 - j] = temp;
}
}
}

void print_matrix(const int mat[N][N]) {
for (int i = 0; i < N; i++) {
for (int j = 0; j < N; j++) printf("%3d ", mat[i][j]);
printf("\n");
}
}

int main(void) {
int mat[N][N] = {
{1, 2, 3},
{4, 5, 6},
{7, 8, 9}
};

printf("Original 3x3 Matrix:\n");
print_matrix(mat);

rotate_90_clockwise(mat);

printf("After 90-Deg Clockwise Rotation:\n");
print_matrix(mat);

return 0;
}
```

>> Solution to Question 22: Write an ANSI C program to merge two pre-sorted integer arrays into a single sorted array in O(N + M) time.

```
int i = 0, j = 0, k = 0;

while (i < n && j < m) {
if (A[i] <= B[j]) {
C[k++] = A[i++];
} else {
C[k++] = B[j++];
}
}

/* Copy remaining elements */
while (i < n) C[k++] = A[i++];
while (j < m) C[k++] = B[j++];
}

int main(void) {
int cohort1[] = {105, 210, 315, 420};
int cohort2[] = {150, 250, 320, 500, 600};
int merged[9];

merge_sorted_arrays(cohort1, 4, cohort2, 5, merged);

printf("Merged Sorted Gene Identifiers: ");
for (int i = 0; i < 9; i++) printf("%d ", merged[i]);
printf("\n");

return 0;
}
```

Solution to Question 24:

>> Explain passing multidimensional arrays to functions with variable dimensions using C99 Variable-Length Array parameters.

```
  for (int j = 0; j < c; j++) {
  printf("%4d ", mat[i][j]);
  }
  printf("\n");
  }
  }

  int main(void) {
  int grid3x2[3][2] = {{1, 2}, {3, 4}, {5, 6}};
  int grid2x4[2][4] = {{10, 20, 30, 40}, {50, 60, 70, 80}};

  print_generic_matrix(3, 2, grid3x2);
  print_generic_matrix(2, 4, grid2x4);

  return 0;
  }
```

Solution to Question 25: Write an ANSI C program to find the second largest element in an array in a single traversal pass in O(N) time.

```
}

int main(void) {
int test_data[] = {45, 12, 89, 70, 89, 23, 67};
int n = sizeof(test_data) / sizeof(test_data[0]);

int sec = find_second_largest(test_data, n);
printf("Array: ");
for (int i = 0; i < n; i++) printf("%d ", test_data[i]);
printf("\nSecond Largest Element: %d\n", sec);

return 0;
}
```

>>Solution to Question 26: an ANSI C program to check whether a given square matrix is an Identity Matrix.

```
};

printf("Matrix I is identity: %s\n", is_identity_matrix(I) ? "TRUE" : "FALSE");
printf("Matrix NonI is identity: %s\n", is_identity_matrix(NonI) ? "TRUE" : "FALSE");
return 0;
}
```

Solution to Question 27: Explain the two-pointer approach for finding a pair of elements in a sorted array that sum to a target value S.

```
if (find_pair_sum(sorted_mw, n, target_mw, &v1, &v2)) {
printf("Found Molecular Weight Pair: %d + %d = %d Da\n", v1, v2, target_mw);
} else {
printf("No matching pair found.\n");
}
return 0;
}
```

>>Solution to Question 28: ANS: Discuss the contribution of mathematical theorems to computer science, including algorithm analysis and optimization.

- Modern CPUs load memory from RAM into L1/L2/L3 caches in discrete 64-byte blocks called Cache Lines.
- Spatial Locality: Accessing address X automatically loads X+1, X+2, ..., X+15 into high-speed cache.

2. Row-Major Traversal (Stride-1, High Spatial Locality):

```
for (int i = 0; i < ROWS; i++)
for (int j = 0; j < COLS; j++)
sum += M[i][j];
```

- Successive memory accesses are adjacent in RAM.
- Accessing M[i][0] fetches M[i][0] through M[i][15] into L1 cache. The next 15 loop iterations hit the L1 cache in ~1 nanosecond!

3. Column-Major Traversal (Strided, Cache Trashing):

```
for (int j = 0; j < COLS; j++)
for (int i = 0; i < ROWS; i++)
sum += M[i][j];
```

- Successive accesses jump across entire rows (offset of COLS * 4 bytes).
- If COLS is large (e.g. 4096 ints = 16 KB), every single access hits a different cache line, incurring an L3 cache miss or RAM access penalty (~50-100 nanoseconds)!
- Row-major traversal is often 10x to 20x faster than column-major traversal in C!

>> Solution to Question 30: Write an ANSI C program to sort biological DNA fragments by molecular length using Insertion Sort.

```
  printf("Unsorted Band Sizes (bp): ");
  for (int i = 0; i < n; i++) printf("%d ", gel_bands[i]);
  printf("\n");

  sort_dna_fragments(gel_bands, n);

  printf("Sorted Ladder Sizes (bp): ");
  for (int i = 0; i < n; i++) printf("%d ", gel_bands[i]);
  printf("\n");

  return 0;
  }
```
